CMSC 420: Hash Functions and Tables

Justin Wyss-Gallifent

January 20, 2026

1 Problems **2**

1 Problems

1. Suppose $K = \mathcal{Z}$ and T has $m = 7$ entries (hence indexed by $\mathcal{Z}_7$). Define the hash function $h : K \to \mathcal{Z}_7$ by $h(x) = ax \mod m$ for $a = 2$. Suppose we are using closed addressing with linked lists and we set $\lambda_{min} = 0.2$ and $\lambda_{max} = 0.8$. We insert the following integers into the hash table:

$$4, 101, 10, 18, 17, 13, 2, 37$$

This should require exactly one rebuild. When the rebuild happens set the new m accordingly and set a to be the smallest integer greater than 1 which is coprime to the new m.

(a) Show the hash table (and its linked lists) after the last insert which does not violate the load factor maximum. Explain why there is no violation.

Solution:
The load factor will be violated when $n/m = n/7 > 0.8$ so $n > 5.6$ so we'll be able to insert 5 elements with no issues,
Observe that:

$$h(\{4, 101, 10, 18, 17\}) = 2\{4, 101, 10, 18, 17\} \mod 7 = \{1, 6, 6, 1, 6\}$$

We thus get linked lists:

- $T[1] \to (4) \to (18)$
- $T[6] \to (101) \to (10) \to (17)$

There is no violation because $n/m = 5/7 \leq 0.8$.

(b) Show the hash table (and its linked lists) after the last insert which does violate the load factor maximum. Explain why there is a violation.

Solution:
Observe that:

$$h(13) = 2(13) \mod 7 = 5$$

We thus get linked lists:

- $T[1] \to (4) \to (18)$
- $T[5] \to (13)$
- $T[6] \to (101) \to (10) \to (17)$

There is a violation because $n/m = 6/7 > 0.8$.

(c) Show the hash table after the final insert. Explain why there is no violation.

Solution:

Observe that:

$$h(\{2, 37\}) = 5\{2, 37\} \mod 12 = \{10, 5\}$$

We thus get linked lists:

- $T[1] \to (101) \to (17)$
- $T[2] \to (10)$
- $T[5] \to (13) \to (37)$
- $T[6] \to (18)$
- $T[8] \to (4)$
- $T[10] \to (2)$

There is no violation because $n/m = 8/12 \leq 0.8$.

2. Suppose $p > 3$ is a prime and we define our hash function $h : \mathcal{Z} \to \mathcal{Z}_p$. The explicit definition of h is not relevant here. To resolve collisions we use cubic probing, meaning if $T[h(x)]$ is full we check $T[h(x)+1^3]$, $T[h(x)+2^3]$, $T[h(x)+3^3]$, and so on with additions taken mod p. We claim that cubic probing will probe at least $\left\lfloor \sqrt[3]{p/3} \right\rfloor + 1$ indices.

   (a) Just to make sure you understand what this means, how many indices are guaranteed to be probed for each of $p = 11, 29, 59, 1009$?

   Solution:
   We have:

   - For $p = 11$ cubic probing will probe at least $\left\lfloor \sqrt[3]{11/3} \right\rfloor + 1 = 2$ indices.
   - For $p = 29$ cubic probing will probe at least $\left\lfloor \sqrt[3]{29/3} \right\rfloor + 1 = 3$ indices.
   - For $p = 59$ cubic probing will probe at least $\left\lfloor \sqrt[3]{59/3} \right\rfloor + 1 = 3$ indices.
   - For $p = 1009$ cubic probing will probe at least $\left\lfloor \sqrt[3]{1009/3} \right\rfloor + 1 = 7$ indices.

   (b) Prove the claim.

   Solution:
   We show that $h(x) + i^3 \neq h(x) + j^3 \mod p$ for $0 \leq i < j \leq \left\lfloor \sqrt[3]{p/3} \right\rfloor$ which will guarantee that there are at least $\left\lfloor \sqrt[3]{p/3} \right\rfloor + 1$ distinct indices hit.
   By way of contradiction assume that there exist $0 \leq i < j \leq \left\lfloor \sqrt[3]{p/3} \right\rfloor$ such that:

   $$h(x) + i^3 = h(x) + j^3 \mod p$$

   It follows that $p \big| (j^3 - i^3)$ and so:

   $$p \big| (j - i)(j^2 + ij + i^2)$$

   This means either $p \big| (j - i)$ or $p \big| (j^2 + ij + i^2)$.
   Since $0 \leq i < j \leq \left\lfloor \sqrt[3]{p/3} \right\rfloor$ we know that $0 \leq j - i < \sqrt[3]{p/3} < p$ (see **note**) and so the first divisibility is not possible.
   Also since $0 \leq i < j \leq \left\lfloor \sqrt[3]{p/3} \right\rfloor$ we know that $0 \leq j^2 + ij + i^2 < \frac{p}{3} + \frac{p}{3} + \frac{p}{3} = p$ and so the second divisibility is not possible.
   Note: There's a special case here that if $p = 3$ then $\sqrt[3]{p/3} = p$ so the strict inequality doesn't hold. But it's true for $p = 3$ too because it will probe all of $0, 1, 2$ which can be easily checked.

3. Suppose $h : K \to \mathcal{Z}_m$ is a hash function which has the property that each index in $\mathcal{Z}_m$ is equally likely to be hit. Assume we are using closed addressing. Suppose n items have been hashed.

   (a) How many empty indices are expected in T? Justify.

   Solution:
   After 1 item is hashed there is a $\frac{1}{m}$ probability that a specific index is hit and hence a $1 - \frac{1}{m}$ probability that a specific index is empty Since separate hashes are independent the probability that a specific index is empty after n items are hashed is:

$$\left(1 - \frac{1}{m}\right)^n$$

   Since this is true for all m indices it follows that the expected number of empty indices is:

$$m\left(1 - \frac{1}{m}\right)^n$$

   (b) Suppose $m = 100$. Use your answer to (a) to determine the largest n value such that you would expect at least half the indices to be empty.

   Solution:
   At least half being empty implies:

$$\begin{aligned} m\left(1 - \frac{1}{100}\right)^n &\geq 0.5m \\ \left(1 - \frac{1}{100}\right)^n &\geq 0.5 \\ \left(\frac{99}{100}\right)^n &\geq 0.5 \\ n \leq \log_{99/100} 0.5 \\ n &\leq 68.97 \end{aligned}$$

   Thus $n = 68$ is it.
   Note: The inequality switches because $\log_{99/100}$ is a decreasing function so $a \leq b$ iff $\log_{99/100} a \geq \log_{99/100} b$.

4. Suppose for load-balancing our hash table we use $\lambda_{min} = 0$ and $\lambda_{max} = 0.5$. We start our hash table with length 1. Suppose we do a sequence of n inserts and each takes time 3. Prove via the aggregate method of amortized analysis that insertion is average case $\mathcal{O}(1)$. Assume that an entire rebuild takes time equal to length of the new table.

Solution:
When we rebuild a table with j elements it will be to length:

$$\frac{2j}{0+0.5} = 4j$$

Our insert process for n items takes time $3n$ for the inserts and we will have a sequence of rebuilds with the following sum of lengths:

$$4 + 16 + 64 + ... + 4^k$$

This is not as easy to understand though, because rebuilds occur when $n > 0.5m$. Let's look for a pattern:
Insert $n = 1$: $1 > 0.5(1)$ so rebuild to length $4(1) = 4$.
Insert $n = 2$: $2 \not> 0.5(4)$ so no rebuild.
Insert $n = 3$: $3 > 0.5(4)$ so rebuild to length $4(3) = 12$.
Insert $n = 4, 5, 6$: $n \not> 0.5(12)$ so no rebuild.
Insert $n = 7$: $7 > 0.5(12)$ so rebuild to length $4(7) = 28$.
Insert $n = 8, ..., 14$: $n \not> 0.5(28)$ so no rebuild.
Insert $n = 15$: $15 > 0.5(28)$ so rebuild to length $4(15) = 60$.
Insert $n = 16, ..., 30$: $n \not> 0.5(60)$ so no rebuild.
Etc.
We see that for each of $i = 1, 2, 3, 4, ...$ that a rebuild occurs when $n = 2^i - 1$ and that rebuild will be to length $4(2^i - 1)$.
It follows that for any n the most recent rebuild will be to size $4(2^k - 1)$ where k is the largest integer satisfying $2^k - 1 < n$. This is equivalent to the largest integer k with:

$$k < \lg(n+1)$$

Because of the strict inequality we can't just floor. Rather this is satisfied by taking the ceiling and subtracting 1:

$$k = \lceil \lg(n+1) \rceil - 1$$

The total cost will then be:

$$
\begin{aligned}
C(n) &= 3n + \sum_{i=1}^{\lceil \lg(n+1) \rceil - 1} 4(2^i - 1) \\
&= 3n + 4 \sum_{i=1}^{\lceil \lg(n+1) \rceil - 1} 2^i - 1 \\
&< 3n + 4 \sum_{i=1}^{\lceil \lg(n+1) \rceil - 1} 2^i \\
&< 3n + 4 \left(2^{\lceil \lg(n+1) \rceil} - 1 - 1\right) \\
&< 3n + 4 \left(2^{\lceil \lg(n+1) \rceil} - 2\right) \\
&< 3n + 4 \left(2^{1+\lg(n+1)} - 2\right) \\
&< 3n + 4 \left(2(n+1) - 2\right)
\end{aligned}
$$

The amortized cost is then also bounded:

$$
AC(n) \leq \frac{1}{n} \left(3n + 4 \left(2(n+1) - 2\right)\right)
$$

The result follows.

5. The following is a really basic (and pretty terrible) simple version of the SHA-256 hash function. It only works on a key x which is a binary string whose length is a multiple of 4.

   (a) Break x up into N sub-keys of length 4 bits: x_1, x_2, ..., x_N.

   (b) Initialize $H = \texttt{0101}$ and $J = \texttt{0111}$.

   (c) For each sub-key x_i do the following:

      i. Update H by XORing it with x_i.

      ii. Update H by XORing it with J.

      iii. Left-rotate J by 1 bit.

   (d) The output is the resulting H, so $h(x) = H$.

   (a) Calculate $h(\texttt{11110100})$. Show the value of H after each step.

   Solution:
   We have: $\texttt{0101} \to \texttt{1101} \to \texttt{0111}$
   The result is 0111.

   (b) Calculate $h(\texttt{00000000})$. Show the value of H after each step.

   Solution:
   We have: $\texttt{0101} \to \texttt{0010} \to \texttt{1100}$
   The result is 1100.

   (c) Find the smallest 8-bit binary string which can be prepended to 10101010 so that the output starts with at exactly two 0s. Do this any way you wish but justify your approach.

   Solution:
   If the 8-bit binary string is 00001000 then the message is 0000100010101010 and H progresses as follows:
   $\texttt{0101} \to \texttt{0010} \to \texttt{0100} \to \texttt{0011} \to \texttt{0010}$
   The result is 0010.
   I did this by writing Python code to do the hashing and then trial-and-error.

6. Suppose you decide to store your data in a hash table with only three entries using closed addressing using standard binary search trees. The keys are integers and the hash function you choose is:

$$h : \mathcal{Z} \to \mathcal{Z}_3 \text{ given by } h(x) = x + 1 \mod 3$$

You insert the following keys in the order given: $1, 5, 3, 7, 10, 9, 4, 2$

Show what the resulting structure looks like. You can illustrate this however you like as long as it is clear.

Solution:
First observe that

$$\begin{aligned} h(1) &= 2 \\ h(5) &= 0 \\ h(3) &= 1 \\ h(7) &= 2 \\ h(10) &= 2 \\ h(9) &= 1 \\ h(4) &= 2 \\ h(2) &= 0 \end{aligned}$$

thus $5, 2$ go into $T[0]$ and $3, 9$ go into $T[1]$ and $1, 7, 10, 4$ go into $T[2]$. The three binary search trees pointed to by $T[0]$, $T[1]$, and $T[2]$ are:

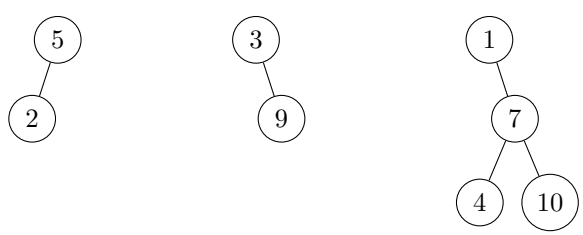

Figure 1: The Binary Search Trees Pointed to by the Hash Entries

7. Suppose a hash table has $m = 150$, $\lambda_{min} = 0$, and $\lambda_{max} = 0.73$. We reach the minimum n necessary to rebuild the table. What will the new value of m be? Simplify your answer.

Solution:
We rebuild the table when we have:

$$\frac{n}{150} > 0.73$$
$$n > 109.5$$

Thus when $n = 110$ the rebuild occurs.
The size of the rebuild is then:

$$\left\lceil \frac{2(110)}{0 + 0.73} \right\rceil = \left\lceil \frac{220}{0.73} \right\rceil = 302$$

8. Suppose for load-balancing our hash table we use $\lambda_{min} = 0$ and $\lambda_{max} = 1/\alpha$ where $\alpha \geq 2$ is an unknown integer. We start our hash table with length $m = 1$. Suppose we do a sequence of n insertions where $n \geq 1$ is fixed.

   (a) For an unknown number of keys n' if we rebuild, what will the corresponding m' be? Simplify.

   Solution:
   It will be:
   $$m = \left\lceil \frac{2n'}{0 + 1/\alpha} \right\rceil = \lceil 2\alpha n' \rceil = 2\alpha n'$$

   (b) Denote by n_i the ith smallest value for which a rebuild occurs. Calculate n_1, n_2, n_3, n_4 and the corresponding m_i for each. Show work and be organized.

   Solution:
   We have:

   - Our first rebuild occurs when $\frac{n'}{1} > \frac{1}{\alpha}$ or $n' > (1/\alpha)$ so at $n' = 1$. Thus $n_1 = 1$. The rebuild will be to length $m_1 = 2\alpha(1) = 2\alpha$.
   - Our second rebuild occurs when $\frac{n'}{2\alpha} > \frac{1}{\alpha}$ or $n' > 2$ so at $n' = 3$. Thus $n_2 = 3$. The rebuild will be to length $m_2 = 2\alpha(3) = 6\alpha$.
   - Our third rebuild occurs when $\frac{n'}{6\alpha} > \frac{1}{\alpha}$ or $n' > 6$ so at $n' = 7$. Thus $n_3 = 7$. The rebuild will be to length $m_3 = 2\alpha(7) = 14\alpha$.
   - Our fourth rebuild occurs when $\frac{n'}{14\alpha} > \frac{1}{\alpha}$ or $n' > 14$ so at $n' = 15$. Thus $n_4 = 15$. The rebuild will be to length $m_4 = 2\alpha(15) = 30\alpha$.

   (c) For an arbitrary $i \geq 1$ what would n_i be and what would the corresponding m_i be? You don't need to prove these - they should be fairly clear from your answers above.

   Solution:
   The ith rebuild occurs when $n_i = 2^i - 1$ to length $m_i = 2(2^i - 1)\alpha$.

(d) Suppose insertion costs 1 and rebuilding cost equals the new length allocated. Write down the expression for the total cost required for all n insertions. Simplify but do not evaluate the sum. If your answer has a floor or ceiling, explain why it's there.

Solution:

The most recent rebuild will be the smallest k with $n_k = 2^k - 1 \leq n$ so $k \leq \lg(n+1)$ so $k = \lfloor \lg(n+1) \rfloor$. Keeping in mind that when we rebuild we need to re-insert, the cost will be:

$$\begin{aligned} C(n) &= n + \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} [m_i + n_i] \\ &= n + \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} \left[\left(2(2^i - 1)\alpha\right) + \left(2^i - 1\right) \right] \\ &= n + (2\alpha + 1) \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} (2^i - 1) \end{aligned}$$

(e) Evaluate the sum.

Solution:

$$\begin{aligned} C(n) &= n + (2\alpha + 1) \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} (2^i - 1) \\ &= n + (2\alpha + 1) \left[\sum_{i=1}^{\lfloor \lg(n+1) \rfloor} 2^i - \sum_{i=1}^{\lfloor \lg(n+1) \rfloor} 1 \right] \\ &= n + (2\alpha + 1) \left(2^{\lfloor \lg(n+1) \rfloor + 1} - 1 \right) + \lfloor \lg(n+1) \rfloor \end{aligned}$$

9. Here is the pseudocode for the hash function from the suggested problems with a `print` statement added. You can assume the length of `x` is a multiple of 4.

```
function hash(x):
    H = 0101
    J = 0111
    break x into n 4-bit chunks x[1],...,x[n]
    for each i = 1 to n inclusive:
        H = H xor x[i]
        H = H xor J
        print H
        J = left rotate J by 1 bit
    end for
    return H
end function
```

(a) Suppose we call `hash(10110101)`. The print statement will execute twice. What will be printed?

First Time	1001
Second Time	0010

Scratch work; not graded:

During the first iteration we have `H = 0101 xor 1011 = 1110` and then `H = 1110 xor 0111 = 1001` and then `J=1110`.
During the second iteration we have `H = 1001 xor 0101 = 1100` and then `H = 1100 xor 1110 = 0010`.

(b) Find a 4-bit string `x` so `hash(x)` prints `1000`. Explain your process.

Solution:

Observe that:

- The loop only runs once with `x[1] = x`.
- We know that the line `H = H xor J` is effectively `H = H xor 0111` so for that line to assign `H = 1000` we must have `H = 1111` before that line.
- The line `H = H xor x[1]` must then be `H = 0101 xor x` so for that line to assign `H = 1111` we must have `x = 1010`.

Thus an answer is `x = 1010`.

10. Suppose a hash table has $m = 200$, $\lambda_{min} = 0$, and $\lambda_{max} = 0.5$. We reach the minimum n necessary to rebuild the table to a new size m. What will n and the new m be? Simplify your answers.

n	101
New m	404

Scratch work; not graded:

We rebuild the table when $n/200 > 0.5$ and so $n > 100$ so $n = 101$. The size of the rebuild is then $\lceil 2(101)/(0 + 0.5) \rceil = \lceil 404 \rceil = 404$.

11. Suppose you decide to store your data in a hash table with only three entries using closed addressing using standard binary search trees. The keys are integers and you are using:

$$h : \mathcal{Z} \to \mathcal{Z}_3 \text{ given by } h(x) = 2x + 1 \mod 3$$

You insert the following keys in the order given: $1, 5, 3, 7, 10, 9, 4, 2$

Show what the resulting structure looks like. You can illustrate this however you like as long as it is clear.

Solution:

First observe that

$$\begin{aligned} h(1) &= 0 \\ h(5) &= 2 \\ h(3) &= 1 \\ h(7) &= 0 \\ h(10) &= 0 \\ h(9) &= 1 \\ h(4) &= 0 \\ h(2) &= 2 \end{aligned}$$

thus $1, 7, 10, 4$ go into $T[0]$ and $3, 9$ go into $T[1]$ and $5, 2$ go into $T[2]$. The three binary search trees pointed to by $T[0]$, $T[1]$, and $T[2]$ are:

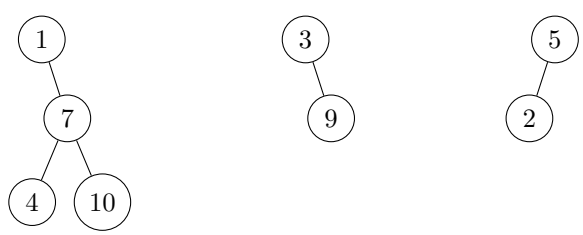

Figure 2: The Binary Search Trees Pointed to by the Hash Entries